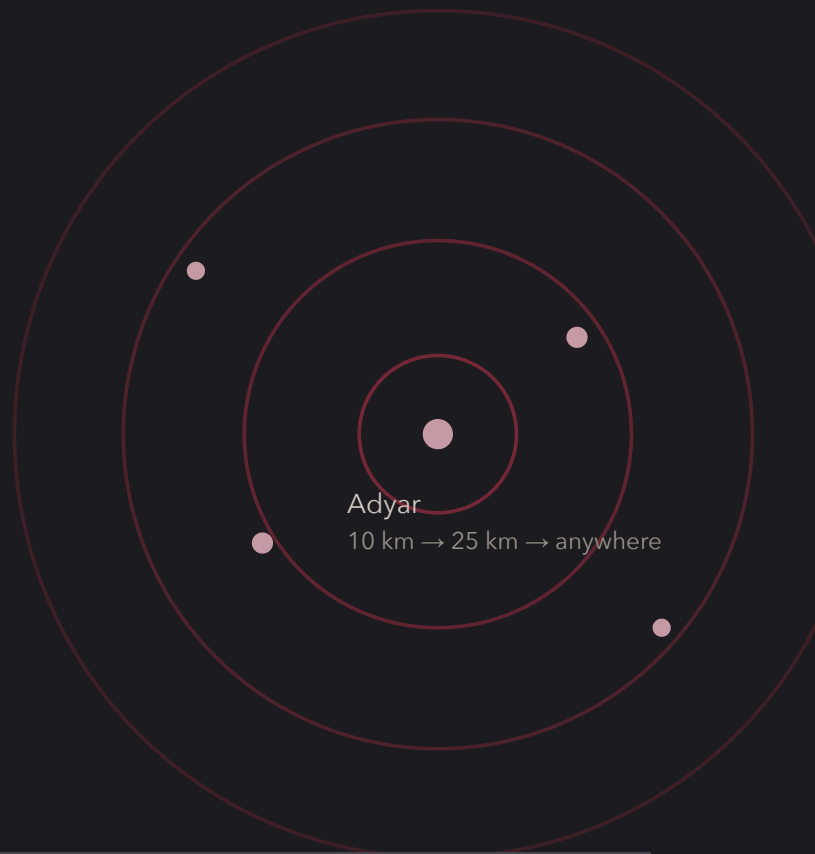


ENGINEERING HANDBOOK

DRAPE

The backend, from A to Z

How a photo becomes a searchable listing, how a sentence becomes a ranked list of outfits nearby, how bookings stay race-proof, and how accounts, sessions and notifications work. Written for developers who are new to the codebase, grounded in the real files, endpoints and SQL.



Contents

1	What DRAPE is	3
	The two core flows, bookings and owner notifications	
2	Architecture at a glance	4
	Browser, one Vercel Function, Supabase, Gemini, OpenAI	
3	The life of a request	5
	Rewrite, router, handler, error wrapper, local dev parity	
4	Data model	7
	Tables, indexes, RLS stance, triggers, the no-overlap constraint	
5	Upload pipeline: photo to listing	9
	Magic bytes, storage, vision, zod + retry, fallbacks, embedding	
6	Search pipeline: sentence to ranked matches	11
	Parser, chips, embedding, match_listings, fallback ladder, reasons	
7	Outfits: “Complete the look”	14
	Vocabulary, slots, the complete_the_look RPC	
8	Bookings and payments	15
	Quote maths, India-time date rules, mock payment, 409 on overlap	
9	Accounts, sessions and Google sign-in	17
	Own users table, script, session cookies, lockout, CSRF, Google via PKCE	
10	Notifications	21
	Trigger to bell, polling	
11	AI providers and rate limits	21
	Models, timeouts, fallbacks, 429 handling	
12	Security summary	22
	Every defence in one place	
13	Operations	23
	Env vars, scripts, tests, deploy, known limitations	
14	Reference	26
	Endpoint table and glossary	

How to read this. Sections 1–3 give you the mental model in ten minutes. Sections 4–10 each follow one feature end to end, from HTTP request to SQL row. Section 14 is the lookup table you will come back to.

Conventions. Paths like `api/_lib/search.ts` are relative to the repo root. Endpoints are written `METHOD /api/path`. `auth` marks endpoints that need a signed-in session; `admin` marks endpoints that need the server key. Secrets are named, never shown.

A note on accounts. Authentication is being rewritten from Supabase Auth to DRAPE’s own users and sessions tables. Section 9 and every “signed in” reference in this guide describe the *new* design.

1 What DRAPE is

DRAPE (“Wear once, don’t buy”) is a hyperlocal, peer-to-peer clothing rental network for Chennai. People who own an outfit they wore once list it; people who need an outfit for one evening describe what they need in plain language and get ranked matches a few kilometres away.

The backend exists to make two flows feel effortless, and to keep them honest:

FLOW A · LIST A PIECE

Snap a photo. The server stores it and asks a vision model to draft the tags (title, category, occasions, style, colours, formality, description). The lister edits the draft, sets size, price per day and pickup area, and publishes. The server embeds the listing so it is searchable immediately.

FLOW B · DESCRIBE WHAT YOU NEED

“Something for my friend’s sangeet, M, under ₹800.” A text model turns that into filters plus a style description; the description is embedded; Postgres filters by size, budget, gender and distance and ranks by vector similarity. If nothing good is close, a fallback ladder relaxes the search and says exactly what changed.

Around these flows sit three supporting features:

- **Bookings.** A borrower picks dates (up to 14 days, up to 90 days ahead) and pays a 20% advance (minimum ₹50). The server recomputes the price; a Postgres exclusion constraint makes double-booking impossible.
- **Owner notifications.** When a booking is inserted, a database trigger writes a notification for the listing’s owner in the same transaction. The owner’s header bell shows it with the borrower’s contact.
- **Complete the look.** For any listing, the API suggests complementary pieces (bottoms for a top, layers for a dress) that share an occasion and a style.

1.1 Design principles you will see everywhere

PRINCIPLE	WHAT IT MEANS IN CODE
Hard filters are SQL, fuzzy intent is vectors	Size, budget, distance and gender are <code>WHERE</code> clauses in <code>match_listings</code> . Occasion, vibe and style only influence cosine ranking. The LLM never decides whether an item fits a budget.
The server never trusts the client	Embeddings, locations, prices, advances and the current user are all computed server-side. Client-sent <code>embedding</code> or <code>location</code> fields are ignored.
AI output is validated, and AI failure is survivable	Every model response goes through a zod schema with one retry. If tagging fails, the lister tags by hand. If the parser fails, search falls back to the raw text. If reasons fail, results still render.
All data access goes through <code>/api</code>	Row-level security is on for every table with no policies, so the public Supabase endpoints return nothing. Only the server, holding the service-role key, reads and writes.
One shared contract	<code>shared/contracts.ts</code> , <code>shared/vocab.ts</code> and <code>shared/booking.ts</code> are imported by both the React app and the API, so shapes cannot drift.

1.2 Repository map (backend-relevant parts)

```
api/  
  router.ts           # the ONLY Vercel Function: route table + dispatch  
  _routes/           # one module per endpoint (underscore = not deployed separately)  
    listings/ index.ts analyze.ts [id]/{index,contact,availability,complete}.ts  
    search/   index.ts reasons.ts  
    bookings/ index.ts mine.ts  
    notifications/ index.ts read.ts  
    auth/ signup.ts login.ts logout.ts providers.ts google/{index,callback}.ts  
    me/index.ts areas.ts health.ts admin/retag.ts  
  _lib/              # shared server code  
    http.ts supabase.ts auth.ts sessions.ts passwords.ts google.ts admin.ts  
    gemini.ts text-llm.ts vision.ts images.ts  
    search.ts search-ai.ts ladder.ts bookings.ts payments.ts  
shared/              # imported by browser AND server  
  vocab.ts contracts.ts booking.ts booking-schema.ts  
supabase/migrations/ # schema, RPCs, triggers, constraints (apply in filename order)  
dev/api-plugin.ts    # serves /api during `npm run dev` via the same router  
scripts/             # seed-listings, retag-listings, verify-prd, create-demo-owner  
tests/               # vitest suites (routing, pipelines, ladder, bookings, contracts)  
vercel.json          # region bom1 + the /api rewrite
```

2 Architecture at a glance

There are only four moving parts: a static single-page app, one serverless function, one Postgres database with file storage, and two AI providers. Everything that needs a secret runs inside the function.

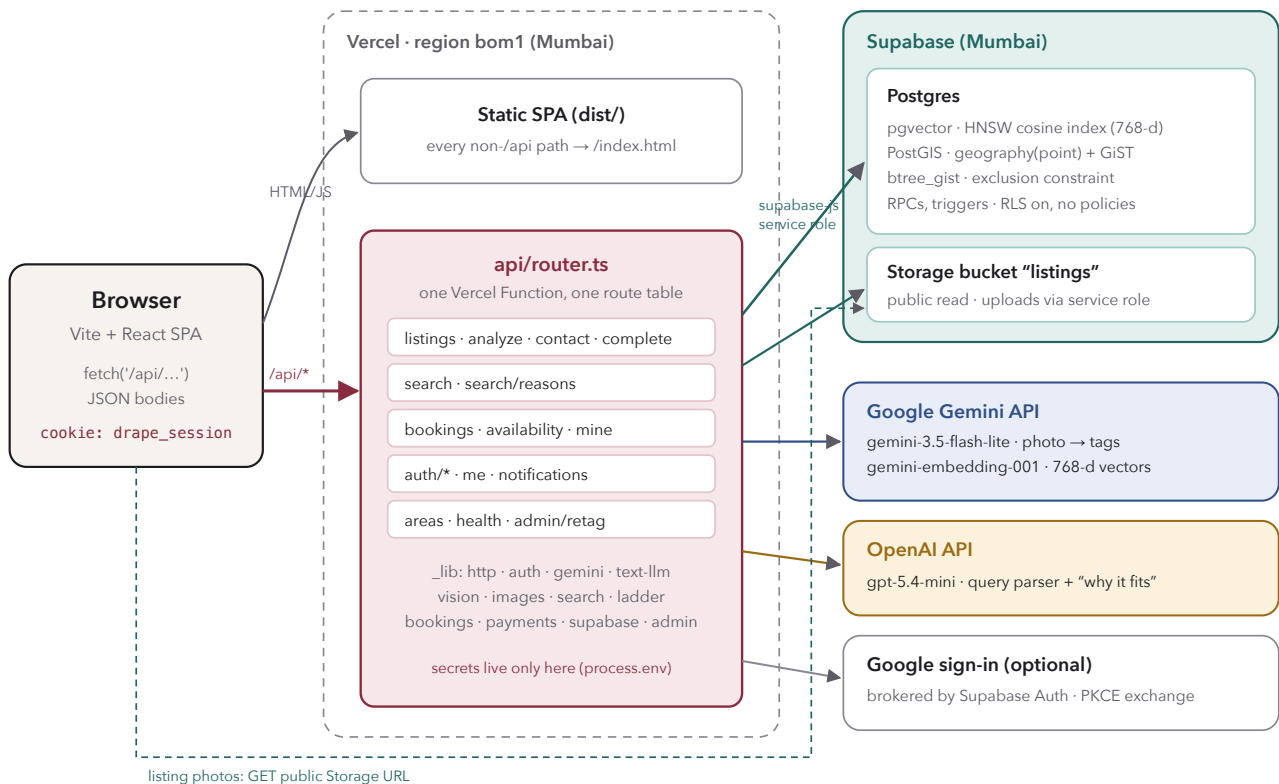


Figure 1. System architecture. The browser only ever talks to Vercel (and fetches listing photos by their public Storage URL). The single function `api/router.ts` holds every secret and is the only client of Postgres, Storage, Gemini, OpenAI and the Supabase Auth token endpoint used for Google sign-in. The project and the database both live in Mumbai (`"regions": ["bom1"]` in `vercel.json`) so each query is a short hop.

2.1 Who does what

COMPONENT	TECHNOLOGY	RESPONSIBILITY
Frontend	Vite + React 19 (<code>src/</code>)	UI only. Calls <code>/api</code> with JSON; holds no secrets. Sessions travel in an <code>httpOnly</code> cookie that JavaScript cannot read.
API	Vercel Function, Web Request → Response handlers, TypeScript, zod	Validation, business rules, orchestration of AI calls, all database access.
Database	Supabase Postgres + <code>vector</code> , <code>postgis</code> , <code>btree_gist</code>	Listings, areas, bookings, users, sessions, notifications; <code>vector</code> + geo search RPCs; triggers and constraints.
File storage	Supabase Storage bucket <code>listings</code>	Listing photos (public read, 8 MB limit, JPEG/PNG/WebP/HEIC/HEIF only).
Vision + embeddings	Gemini REST (<code>api/_lib/gemini.ts</code> , no SDK)	Photo tagging; 768-d embeddings for listings and queries.
Search text	OpenAI Chat Completions (<code>api/_lib/text-llm.ts</code>)	Query parsing and “why it fits” reasons, with Gemini as automatic fallback.

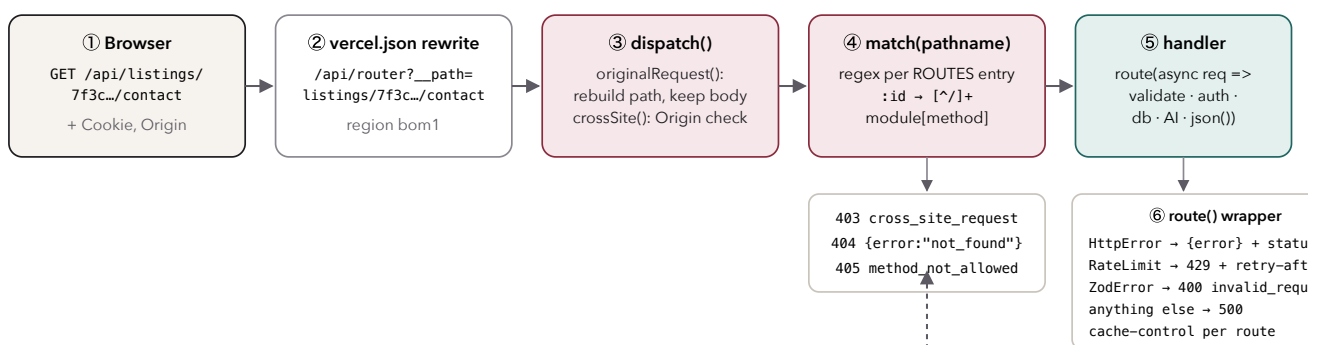
WHY ONE FUNCTION?

Vercel’s Hobby plan allows at most 12 serverless functions per deployment, and DRAPE has more than 20 endpoints. So every route lives in `api/_routes/` (the leading underscore tells Vercel not to deploy those files as functions) and a single `api/router.ts` dispatches to them. A test in `tests/api-plugin.test.ts` fails if anyone adds a second function or forgets to register a route.

3 The life of a request

Every API call follows the same path. Once you know it, any endpoint is easy to trace.

PRODUCTION (Vercel)



LOCAL DEV (npm run dev)

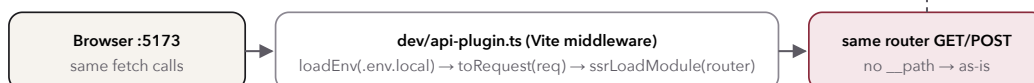


Figure 2. Request lifecycle. Production and local development share steps ③–⑥ exactly; only the way the request reaches the router differs.

3.1 Step by step

1. **Rewrite.** `vercel.json` has two rewrites. `/api/:path(.*)` becomes `/api/router?__path=:path`; everything else becomes `/index.html` so client-side routes work.

2. **Rebuild the original request.** `originalRequest()` in `api/router.ts` reads `__path`, deletes it from the query string, restores `url.pathname = /api/<path>`, and re-creates the `Request` with the same method, headers (including `Cookie` and `Origin`) and body bytes. Handlers therefore see their real URL and query string.
3. **Cross-site check.** `crossSite()` rejects any non-GET request whose `Origin` is another site with `403 {"error": "cross_site_request"}` (Section 9.5).
4. **Match.** `ROUTES` is an ordered list of `[pattern, module]`. Each pattern compiles to `^pattern/?$` with `:id` replaced by `[^/]+`. Order matters: static segments come before `:id`, so `/api/listings/analyze` never matches `/api/listings/:id`. No match gives `404 {"error": "not_found"}`; a module without that method export gives `405 {"error": "method_not_allowed"}`.
5. **Handle.** Each route module exports `GET` and/or `POST`, each wrapped in `route()` from `api/_lib/http.ts`. Handlers throw instead of building error responses. JSON bodies are read with `readJson()`, which rejects any body not sent as `Content-Type: application/json` with `415` and malformed JSON with `400`.
6. **Wrap errors.** `route()` turns a thrown `HttpError(status, message)` into `{"error": message}` with that status; a `RateLimitError` (a subclass carrying `retryAfterSeconds`) into `429 {"error": "rate_limited", "message", "retry_after"}` plus a `Retry-After` header; a `ZodError` into `400 {"error": "invalid_request", "issues": [...]}`; anything else into a logged `500 {"error": "internal_error"}`. Stack traces never reach the client.
7. **Respond.** `json(data, status)` sets `content-type: application/json` and `cache-control: no-store`. Public, non-personal reads opt in to CDN caching with `publicCache(response, seconds)`, which sets `public, max-age=0, s-maxage=N, stale-while-revalidate=10N`: `/api/areas` 1 h, `/api/listings` 30 s, `/api/listings/:id` and `/:id/complete` 60 s. Everything personal or date-sensitive (search, contact, availability, bookings, me, notifications, auth) stays `no-store`.

3.2 Anatomy of a route module

```
// api/_routes/listings/[id]/contact.ts
export const GET = route(async (request) => {
  const id = listingIdFrom(request) // zod uuid check, else 404 without touching the DB
  const { data, error } = await db().from('listings')
    .select('owner_name, owner_contact').eq('id', id).maybeSingle()
  if (error) throw error // becomes 500 internal_error
  if (!data) throw new HttpError(404, 'Listing not found')
  return json(data)
})
```

3.3 Local development parity

`npm run dev` starts Vite with the plugin in `dev/api-plugin.ts`. The plugin copies every variable from `.env.local` into `process.env` (mirroring Vercel's server-side env), converts Node's `IncomingMessage` into a `Web Request`, loads `api/router.ts` through `server.ssrLoadModule` on every request (so edits hot-reload), and calls the router's `GET` or `POST`. Because no `__path` is present, the router uses the URL as-is. The whole stack therefore runs locally without `vercel dev`.

ADDING AN ENDPOINT

1) Create `api/_routes/<name>.ts` exporting `GET / POST` wrapped in `route()`. 2) Import it in `api/router.ts` and add a `[pattern, module]` row, with static segments above any `:id` sibling. 3) Server imports must end in `.js` (ESM on Vercel); `tests/esm-imports.test.ts` checks this. `tests/api-plugin.test.ts` fails if a route file is not registered.

4 Data model

The schema lives in `supabase/migrations/` and is applied in filename order (SQL editor or `supabase db push`). Extensions are installed into the `extensions` schema per Supabase convention: `vector` (pgvector), `postgis` and `btree_gist`.

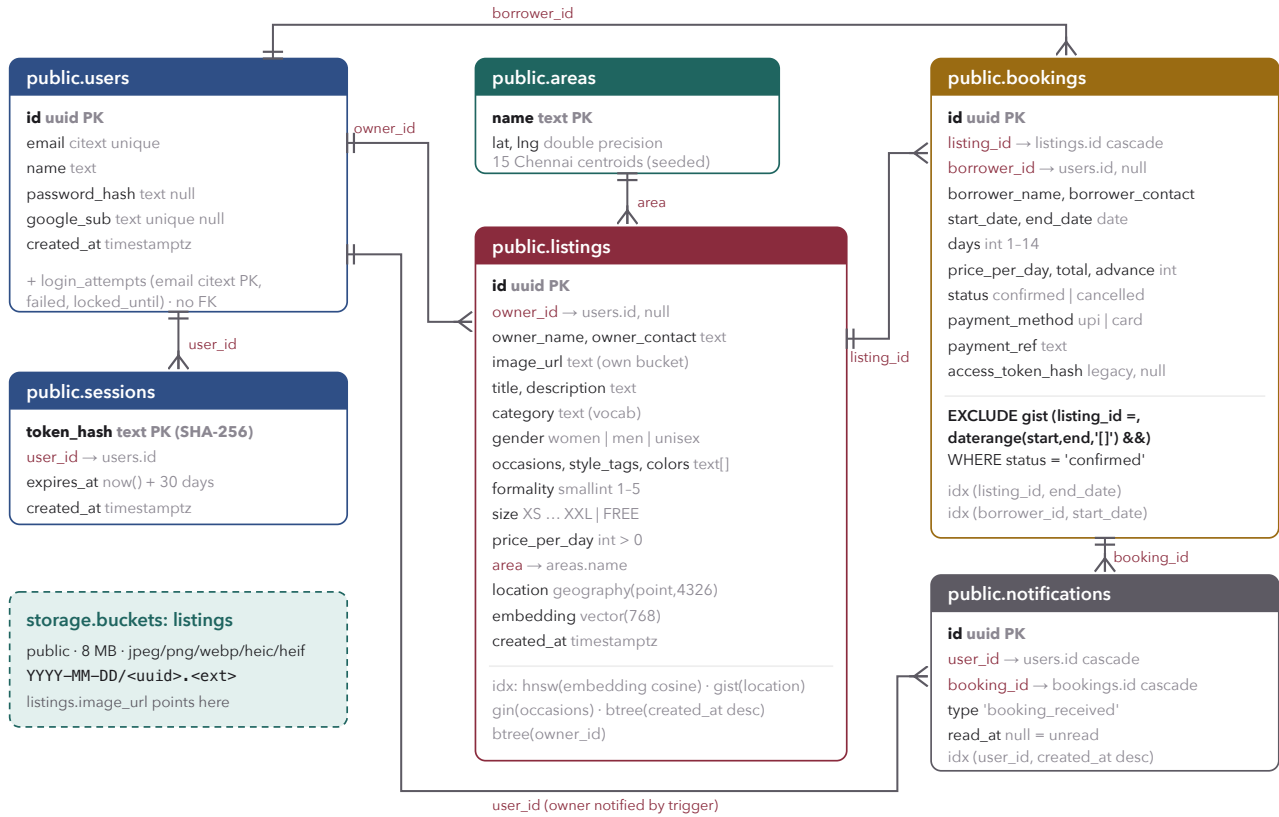


Figure 3. Entity-relationship view. Blue tables are the new account tables (Section 9); the `owner_id`, `borrower_id` and `user_id` foreign keys reference `public.users`. Crow's feet mark the "many" side.

4.1 Tables

TABLE	CREATED IN	PURPOSE AND NOTABLE DETAILS
areas	...0929000001_schema.sql, seeded by ...0929000002_seed_areas.sql	15 Chennai neighbourhood centroids (T. Nagar, Adyar, Velachery, Anna Nagar, Tambaram, OMR, ...). The area picker, the search origin and each listing's location all come from here. There is no real geolocation.
listings	...0929000001_schema.sql; owner_id added later	One rentable piece. AI-drafted fields plus lister-set size, price_per_day, area, owner name/contact. location and embedding are always computed by the server. owner_contact is never in the public column list.
bookings	...0930000001_bookings.sql	Reserved date ranges with the recomputed price, advance and mock payment reference. Check constraints mirror the business rules (days between 1 and 14, advance <= total, end_date >= start_date). access_token_hash is a Phase 6 leftover, now nullable.
notifications	...0930000002_accounts_notifications.sql	The header bell. Written only by a trigger. type is constrained to 'booking_received' for now.

TABLE	CREATED IN	PURPOSE AND NOTABLE DETAILS
<code>users</code> , <code>sessions</code>	<code>...0930000004_own_auth.sql</code> (Section 9)	DRAPE's own accounts and server-side sessions. They replace Supabase Auth's <code>auth.users</code> and the Phase 7 <code>profiles</code> table (whose name now lives in <code>users.name</code>).

4.2 Indexes and why each exists

- `listings_embedding_idx`: **HNSW** on `embedding vector_cosine_ops`. Makes `order by embedding <=> q_emb` an approximate nearest-neighbour scan instead of a full sort.
- `listings_location_idx`: **GiST** on the geography point. Serves `st_dwithin` radius filters.
- `listings_occasions_idx`: **GIN** on `occasions`. Serves `GET /api/listings?occasion=`, which uses array containment (`@>`). The GiST location index also serves the `<=>` distance ordering in `nearby_listings`.
- `listings_created_at_idx`, `listings_owner_idx`: newest-first lists and profile stats.
- `bookings_listing_dates_idx` (`listing_id`, `end_date`): availability look-ups (“upcoming bookings of this listing”). `bookings_borrower_idx` (`borrower_id`, `start_date`): My Rentals.
- `notifications_user_idx` (`user_id`, `created_at desc`): the bell's newest-first query.

4.3 Row-level security: on, with no policies

Every table has `enable row level security` and **no policies**. In Postgres that means the `anon` and `authenticated` roles can read and write *nothing* through Supabase's public REST or GraphQL endpoints. Only the service role (which bypasses RLS) can touch data, and only the API holds the service-role key. The RPCs follow the same rule: `revoke execute ... from public, anon, authenticated` and `grant execute ... to service_role`. The consequence is simple: **the only way to data is through `/api`**, where validation and authorisation live.

4.4 The constraint that makes double-booking impossible

```
create extension if not exists btree_gist with schema extensions;
constraint bookings_no_overlap exclude using gist (
  listing_id with =,
  daterange(start_date, end_date, '[]') with &&
) where (status = 'confirmed')
```

An exclusion constraint says “no two rows may have *equal* `listing_id` **and** *overlapping* date ranges”. `btree_gist` is what lets a GiST index handle the plain equality on a uuid alongside the range overlap. The range is inclusive on both ends (`[]`), so returning on the 12th and someone else picking up on the 12th *is* a conflict. Because the database enforces this atomically, two borrowers paying at the same instant cannot both win: the second insert fails with `SQLSTATE 23P01`, which the API maps to HTTP 409. Cancelled bookings are excluded from the check by the partial `where`.

4.5 Functions and triggers

OBJECT	WHAT IT DOES
<code>match_listings(q_emb, q_size, q_max_price, q_gender, q_lat, q_lng, q_radius_km, k)</code>	Search RPC (<code>language sql stable</code>). Applies hard filters, then orders by cosine distance. Section 6.
<code>nearby_listings(q_lat, q_lng, q_occasion, lim, off)</code>	Browsing RPC (<code>...09300000005_nearby_listings.sql</code>). Optional occasion filter (<code>occasions @> array[q_occasion]</code>), ordered nearest first by <code>location <=> point</code> then newest, with <code>distance_km</code> and a <code>count(*) over ()</code> total for paging. Section 4.6.

OBJECT	WHAT IT DOES
<code>complete_the_look(anchor, pair_categories, k)</code>	Outfit-pairing RPC. Section 7.
<code>notify_owner_of_booking()</code> + <code>trigger</code> <code>on_booking_created</code>	after <code>insert</code> on <code>bookings</code> : inserts a <code>booking_received</code> notification for the listing's owner. security definer with <code>search_path = ''</code> (fully-qualified names, no search-path hijack). Section 10.

4.6 Browsing: nearest first, with paging

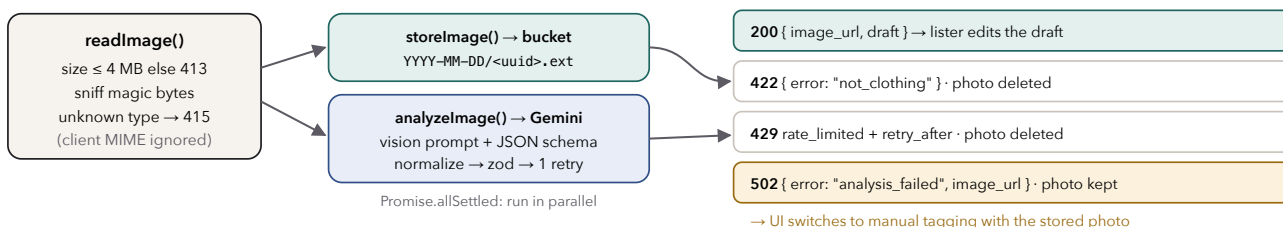
The home “nearby” row and Explore use `GET /api/listings?area=&occasion=&limit=&offset=` rather than search. With an `area`, the route calls `nearby_listings` with that area’s centroid, and results come back nearest first with `distance_km`. The RPC’s window count (`count(*) over ()`) becomes `total`, so the client knows when to stop paging. Explore pages 24 at a time. Without an `area`, the route falls back to newest first, with `.range(offset, offset + limit - 1)` and an exact count.

```
GET /api/listings?area=Adyar&occasion=wedding&limit=24&offset=0
200 cache-control: public, max-age=0, s-maxage=30, stale-while-revalidate=300
{ "listings": [ { "id": "...", "title": "...", "area": "Besant Nagar", "distance_km": 0.3, ... } ], "total": 17 }
```

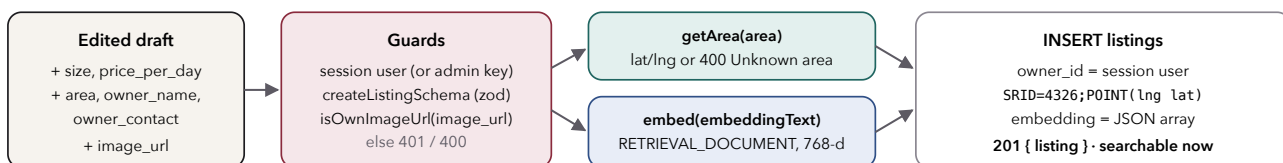
5 Upload pipeline: photo to listing

Listing is a two-request conversation. `POST /api/listings/analyze` turns a photo into an editable draft; after the human edits it, `POST /api/listings` publishes it. Splitting them keeps a human in the loop: the model proposes, the lister decides.

REQUEST 1 · `POST /api/listings/analyze` (multipart “image”, signed in)



REQUEST 2 · `POST /api/listings` (JSON, signed in)



Both routes: `api_routes/listings/analyze.ts` and `api_routes/listings/index.ts` · helpers in `api_lib/images.ts`, `vision.ts`, `gemini.ts`

Figure 4. The upload pipeline. Storage and tagging run in parallel to keep the wait under the PRD’s 10-second target (tagging alone measures about 3 s).

5.1 Step 1: check the caller, then read and sniff the image

Analyze requires a signed-in session (`401` otherwise), or the server key in `x-admin-key` for the seed and readiness scripts, because every call spends AI quota and stores a photo. The UI therefore asks the lister to sign in before choosing a photo. The UI downsizes photos before upload, because Vercel rejects request bodies over 4.5 MB. The

server enforces `MAX_IMAGE_BYTES = 4 MB` (413 otherwise). It never trusts the browser's declared MIME type: `sniffImage()` in `api/_lib/images.ts` reads the first bytes.

TYPE	MAGIC BYTES CHECKED	STORED AS
JPEG	FF D8 FF at offset 0	.jpeg , image/jpeg
PNG	89 then ASCII PNG	.png , image/png
WebP	RIFF at 0 and WEBP at 8	.webp , image/webp
HEIC/HEIF	ftyp at 4, brand heic heix hevc mif1 msf1 heif at 8	.heic , image/heic

Anything else is `415 Unsupported image type`, before any model is called.

5.2 Step 2: store and tag in parallel

`storeImage()` uploads to the `listings` bucket at `YYYY-MM-DD/<random uuid>.<ext>` (`upsert: false`) and returns the public URL. At the same time `analyzeImage()` in `api/_lib/vision.ts` sends the image inline (base64) to Gemini with:

- **A system prompt with strict rules** (`VISION_SYSTEM_PROMPT`): if the photo is not a rentable garment, return only `{"is_clothing": false}`; describe the garment, never the person, body or background; pick `category`, `occasions` and `gender` only from the controlled vocabulary, with disambiguation notes (e.g. *ethnic_jacket* is Nehru/bandhgala only; western tailored is *blazer*; if the photo shows a whole outfit, tag the most prominent rentable piece); 3–6 lowercase style tags; 1–3 colours; formality 1–5; a two-sentence description whose **second sentence must say where the piece does not fit well** (this makes search honest).
- **Structured output**: `responseMimeType: application/json` with `VISION_RESPONSE_SCHEMA` (enums generated from `shared/vocab.ts`), `temperature 0.2`, `thinkingLevel: minimal`, 20-second timeout.
- **Normalise, then validate**: `normalizeVision()` de-duplicates and lower-cases tags, caps list lengths and clamps formality into 1–5; then `visionResultSchema` (a zod discriminated union on `is_clothing`) validates the result.
- **One retry**: `withOneRetry()` reruns generate + validate once if it throws. It does *not* retry a `RateLimitError`, because an immediate retry would spend another request from an exhausted quota.

5.3 Step 3: decide the outcome

`Promise.allSettled` lets the route inspect both results. A storage failure is a real error. A rate limit deletes the photo and returns 429 so the UI can say “try again in N seconds”. Any other vision failure **keeps** the photo and returns `502 {"error": "analysis_failed", "image_url"}`, so the UI falls back to manual tagging with the already-stored image. A not-clothing verdict deletes the photo and returns 422.

```
// 200 OK – POST /api/listings/analyze
{
  "image_url": "https://<project>.supabase.co/storage/v1/object/public/listings/2026-09-30/5b1e...jpeg",
  "draft": {
    "title": "Maroon silk saree with gold zari border",
    "category": "saree", "gender": "women",
    "occasions": ["wedding", "reception", "festival"],
    "style_tags": ["silk", "zari", "traditional"],
    "colors": ["maroon", "gold"], "formality": 4,
    "description": "A rich silk saree for weddings and receptions. Too heavy for daytime college events."
  }
}
```

5.4 Step 4: publish

`POST /api/listings` requires a signed-in session (seed scripts may instead send the server key; see Section 12). The body is validated by `createListingSchema` (the draft plus `image_url`, `size`, `price_per_day` 1–100000, `area`, `owner_name`, `owner_contact`). `isOwnImageUrl()` rejects any image URL that is not under `<SUPABASE_URL>/storage/v1/object/public/listings/` (and any path containing `..`), so listings can only show photos this app stored.

The server then computes the two fields the client must never send, in parallel:

- **Location from area.** `getArea()` looks up the centroid and the insert writes `location = 'SRID=4326;POINT(lng lat)'`. Note the order: longitude first, as PostGIS expects.
- **Embedding from semantics only.** `embeddingText()` joins title, category, occasions, style, colours and description. Size and price are deliberately excluded because they are SQL filters, not meaning.

```
// embeddingText() output, embedded with taskType RETRIEVAL_DOCUMENT
Maroon silk saree with gold zari border. saree. occasions: wedding, reception, festival.
style: silk, zari, traditional. colors: maroon, gold. A rich silk saree for weddings and
receptions. Too heavy for daytime college events.
```

The row is inserted with `owner_id` set to the session user and returned as `201 {"listing": { ...public columns... }}`. The write is synchronous, so the listing is searchable the moment the response arrives.

6 Search pipeline: sentence to ranked matches

`POST /api/search` is the heart of DRAPE. The code is split into pure, unit-tested pieces: `api/_lib/search-ai.ts` (parser and reasons), `api/_lib/ladder.ts` (filters, fallback ladder, messages) and `api/_lib/search.ts` (orchestration).

```
// Request
POST /api/search
{ "query": "need something for a friend's sangeet, M, under ₹800", "area": "Adyar",
  "size": "M", "max_price": 800, "reasons": false } // size, max_price, reasons optional
```

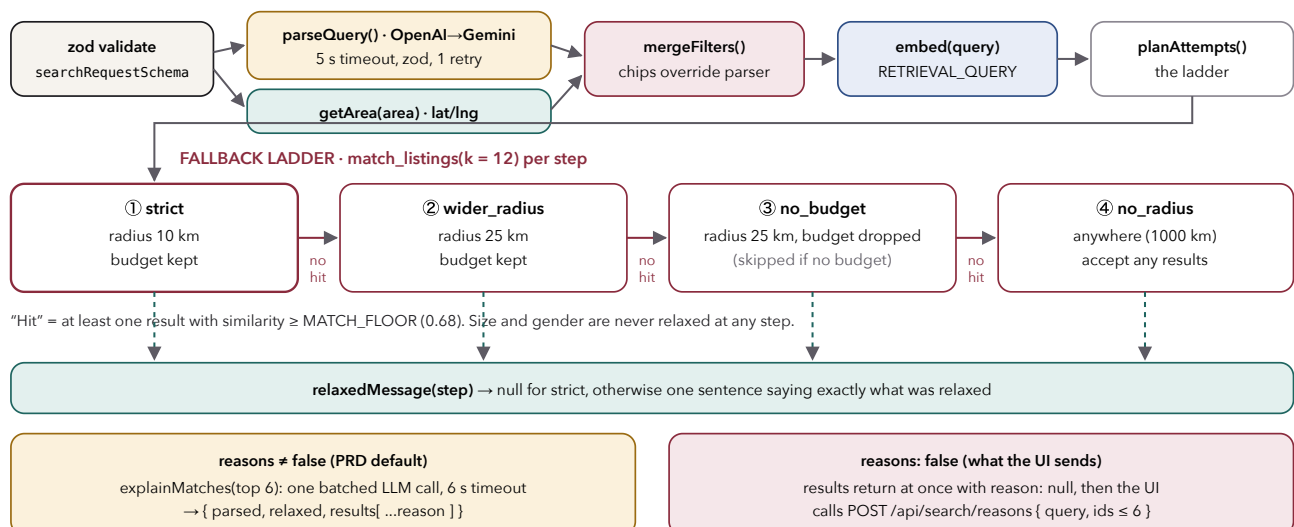


Figure 5. The search pipeline. Every response also carries a `Server-Timing` header with the duration of each stage.

6.1 Parse the sentence into filters

`parseQuery()` sends the raw text to the text LLM with `PARSER_SYSTEM_PROMPT` and a strict JSON schema. It must return exactly five keys, using `null` for anything not clearly stated (“never guess”):

```
{ "occasion": "sangeet_mehendi", "size": "M", "max_price": 800, "gender": null,
  "style_query": "festive, colourful outfit for a sangeet, comfortable to dance in" }
```

The prompt maps slang to vocabulary (“mehendi” → `sangeet_mehendi`, “rave/techno/EDM” → `club_night`, “1k” → 1000) and sets `gender` only when the message clearly says who it is for. `style_query` is the request rewritten as a description of the ideal outfit, with size and price removed. The result is validated by `parsedQuerySchema` with one retry. If the parser still fails, search **degrades gracefully**: all filters become `null` and `style_query` is the raw text.

6.2 Merge with the UI chips

`mergeFilters()`: explicit `size` and `max_price` from the request (the UI’s chips) win over the parser; `gender` always comes from the parser; the `area` always comes from the header selector. The `parsed` object returned to the client reflects the merged values.

6.3 Embed the intent

`queryEmbeddingText()` appends the occasion label to the style query, mirroring how listings are embedded: “festive ... to dance in. occasion: Sangeet / Mehendi”. It is embedded with `gemini-embedding-001`, `taskType: RETRIEVAL_QUERY`, 768 dimensions. The same `embed()` helper serves upload and search and rejects any vector that is not exactly 768-d, so dimensions cannot drift. Note that **occasion is a ranking signal, not a filter**.

6.4 Filter and rank in Postgres

```
select ..., st_distance(l.location, st_makepoint(q_lng, q_lat)::geography) / 1000 as distance_km,
          1 - (l.embedding <=> q_emb) as similarity
from listings l
where (q_size is null or l.size = q_size or l.size = 'FREE')
   and (q_max_price is null or l.price_per_day <= q_max_price)
   and (q_gender is null or l.gender in (q_gender, 'unisex'))
   and (q_lat is null or st_dwithin(l.location, st_makepoint(q_lng, q_lat)::geography, q_radius_km
* 1000))
order by l.embedding <=> q_emb
limit k; -- k = RESULT_LIMIT = 12
```

`<=>` is pgvector’s cosine distance, so `similarity = 1 - distance`. Free-size pieces match any size, unisex pieces match either gender. The embedding is passed as a JSON string (`JSON.stringify(embedding)`), which PostgREST casts to `vector`.

6.5 The fallback ladder and the match-quality floor

Vector search almost always returns *something*, so “zero results” alone would rarely trigger a fallback. Instead a step only counts as a hit if at least one result is a genuinely good match: `isGoodEnough()` requires a similarity $\geq$ `MATCH_FLOOR` (default **0.68**, overridable by env). The last step accepts any non-empty result.

HOW 0.68 WAS CHOSEN

Calibrated on `gemini-embedding-001` over 14 real queries: plausible top matches scored **0.686–0.85**, poor ones **0.59–0.66**. The example that set the line: “wedding guest saree”, whose nearest item was a beige dress at 0.653. The floor sits in the gap. If you change the embedding model, re-calibrate (and re-embed every listing).

`planAttempts()` builds the steps (Figure 5); steps that would repeat the previous query are skipped (no budget step when there is no budget). Size and gender are never relaxed: a dress that does not fit is not a match. The step that produced the results drives `relaxedMessage()` :

STEP USED	MESSAGE (AREA ADYAR, BUDGET ₹800)
<code>strict</code>	<code>null</code> (nothing was relaxed)
<code>wider_radius</code>	No good match under ₹800 within 10 km of Adyar, so here are options up to 25 km away.
<code>no_budget</code>	No good match under ₹800 near Adyar, so here are options up to ₹1,200 within 25 km.
<code>no_radius</code>	No good match under ₹800 within 25 km of Adyar, so here are the closest matches across Chennai, up to ₹1,200.

“Up to ₹1,200” is the highest price among the returned results, formatted with `en-IN` grouping. The wording says “no good match” rather than “nothing”, because the floor can relax a step that technically had weak results. The last step uses a 1000 km radius rather than none, so `distance_km` stays populated.

6.6 “Why it fits”: batched reasons, results first

`explainMatches()` makes **one** LLM call for the top `REASON_COUNT = 6` results (never one per card). The prompt asks for ≤ 12 words per item, specific to fabric, colour, silhouette or vibe; never mention size, price, distance or availability; never assume the wearer’s gender; be honest when an item is a weak fit. The server keeps only reasons whose `id` was in the request and trims each to 15 words (`clampWords()`). If the call fails, results are returned with `reason: null`; a reasons failure never fails a search.

By default (the PRD contract) reasons are inline. The UI sends `"reasons": false` so the grid renders as soon as matching is done (about 2 s), then calls `POST /api/search/reasons` :

```
POST /api/search/reasons { "query": "...sangeet...", "ids": ["7f3c...", "a91d..."] } // 1–6 uuids
200 { "reasons": [ { "id": "7f3c...", "reason": "Bright mirror-work lehenga made for twirling at a
sangeet" } ] }
```

6.7 The response and Server-Timing

```
200 OK
server-timing: parse;dur=980, embed;dur=210, match;dur=140, total;dur=1335
{
  "parsed": { "occasion": "sangeet_mehendi", "size": "M", "max_price": 800, "gender": null,
  "style_query": "...",
  "relaxed": null,
  "results": [ { "id": "7f3c...", "title": "...", "image_url": "...", "category": "lehenga",
    "occasions": [...], "style_tags": [...], "size": "M", "price_per_day": 700,
    "area": "Besant Nagar", "description": "...", "distance_km": 0.4,
    "similarity": 0.74, "reason": null } ]
}
```

The `Timer` class in `api/_lib/search.ts` records `parse`, `embed`, `match` and (when inline) `reasons` laps plus a `total`. Browser dev tools show these in the Network → Timing panel, which is the fastest way to see where a slow search spent its time. (The durations above are illustrative.)

7 Outfits: “Complete the look”

7.1 The controlled vocabulary

`shared/vocab.ts` is the single source of truth for what the vision model may output, what the parser may return, what zod accepts, and what the UI shows. Changing it means updating prompts automatically (they interpolate the lists) and re-tagging old listings with `scripts/retag-listings.mjs`.

LIST	VALUES
Categories (20)	saree lehenga kurta_set salwar_suit gown dress sherwani suit blazer shirt skirt ethnic_jacket top hoodie jacket jeans trousers shorts co_ord_set other
Occasions (12)	wedding reception sangeet_mehendi festival party interview formal_event photoshoot college_event casual_outing club_night concert
Sizes	XS S M L XL XXL FREE
Genders	women men unisex (the parser only ever returns <code>women / men / null</code>)
Constants	EMBEDDING_DIMENSIONS = 768 , DEFAULT_RADIUS_KM = 10

7.2 Slots and pairing rules

Every category has an outfit slot (`CATEGORY_SLOT`), and each slot lists what it pairs with, in priority order (`PAIRS_WITH`). The first entry is what *completes* the piece; later entries are extras.

SLOT	CATEGORIES	PAIRS WITH (PRIORITY ORDER)
one_piece	saree, lehenga, kurta_set, salwar_suit, gown, dress, sherwani, suit, co_ord_set	layer
top	shirt, top, hoodie	bottom, then layer
bottom	skirt, jeans, trousers, shorts	top, then layer
layer	blazer, jacket, ethnic_jacket	top, bottom, one_piece
other	other	nothing (empty result)

One exception: `suit` and `sherwani` already include their own layer, so when the anchor is a layer they are never suggested as the one-piece to wear under it (`categoriesInSlot()` filters them out).

7.3 The endpoint and the RPC

`GET /api/listings/:id/complete` loads the anchor’s category, derives its slot and pairing slots, and calls the RPC once per pairing slot in parallel: `k = 4` for the completing slot, `k = 2` for extras. Results are concatenated in priority order, tagged with their `slot`, and capped at 8.

```
select l.~, st_distance(l.location, a.location) / 1000, 1 - (l.embedding <=> a.embedding)
from listings a join listings l on l.id <> a.id
where a.id = anchor
  and l.category = any(pair_categories)           -- complementary slot
  and (a.gender = 'unisex' or l.gender in (a.gender, 'unisex')) -- gender-compatible
  and l.occasions && a.occasions                  -- share ≥ 1 occasion
order by (l.embedding <=> a.embedding)            -- style distance
        + least(st_distance(l.location, a.location) / 1000, 30) / 300 -- nearness: 30 km costs 0.1
limit k;
```

The ranking is style first with a gentle nearness bonus: distance adds up to 0.1 to the cosine distance, capped at 30 km, so a slightly less similar piece next door can beat a perfect one across town.

```
GET /api/listings/9c1e.../complete // anchor is a black mesh top
200 { "slot": "top", "pairs_with": ["bottom", "layer"],
      "results": [ { "id": "...", "title": "Black leather cargo trousers", "slot": "bottom",
                    "similarity": 0.81,
                    "distance_km": 3.2, ... }, ... up to 4 bottoms, then up to 2 layers ] }
```

The listing page shows these results as a “Complete the look” card row.

8 Bookings and payments

Booking rules live in `shared/booking.ts` so the UI can show a live quote and the server can **recompute** it. The server never trusts a price, advance or day count sent by the browser; the request carries only the listing, the dates, a contact and a payment method.

8.1 Quote maths

```
days    = daysBetween(start, end) + 1 // pickup AND return day both count
total    = days × price_per_day
advance  = min(total, max(MIN_ADVANCE = 50, round(total × ADVANCE_RATE = 0.2)))
due_at_pickup = total - advance
```

WORKED EXAMPLE	DAYS	TOTAL	ADVANCE	DUE AT PICKUP
₹400/day, pick up 10 Oct, return 12 Oct	3	₹1,200	20% = ₹240	₹960
₹150/day, same day (10 → 10 Oct)	1	₹150	20% = ₹30 → minimum ₹50	₹100
₹40/day, same day	1	₹40	min(₹40, ₹50) = ₹40	₹0

These three cases are exactly the assertions in `tests/bookings.test.ts`.

8.2 Date rules, in India time

Dates are plain `YYYY-MM-DD` calendar days with no time of day, so there is no timezone drift. “Today” is computed in `Asia/Kolkata` by `todayInIndia()` (`Intl.DateTimeFormat('en-CA', {timeZone})`), so a request at 20:00 UTC on 29 Sep counts as 30 Sep. Day arithmetic uses `Date.UTC` to avoid DST effects. `validateDates()` returns a human-readable problem or `null`:

RULE	MESSAGE
Both dates match <code>YYYY-MM-DD</code>	Pick a pickup and a return date.
Date exists (no 31 Nov; round-trip through <code>addDays(d, 0)</code>)	That date doesn't exist.
Pickup ≥ today (IST)	Pickup can't be in the past.
Pickup ≤ today + 90 days (<code>MAX_DAYS_AHEAD</code>)	You can book up to 90 days ahead.
Return ≥ pickup	Return must be on or after pickup.
At most 14 days inclusive (<code>MAX_BOOKING_DAYS</code>)	Bookings can be at most 14 days.

8.3 The booking request, end to end

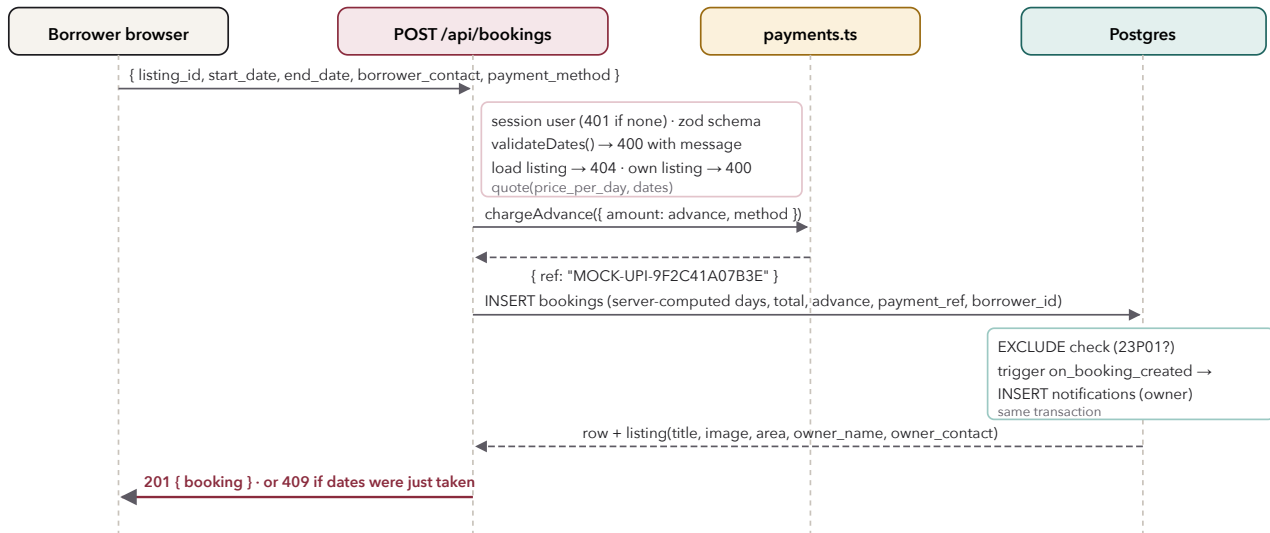


Figure 6. Booking sequence (`api/_routes/bookings/index.ts`). The owner's notification is written by the database, not by the route.

```

201 Created
{ "booking": { "id": "...", "listing_id": "...", "start_date": "2026-10-10", "end_date": "2026-10-12",
  "days": 3, "price_per_day": 400, "total": 1200, "advance": 240, "status": "confirmed",
  "payment_ref": "MOCK-UPI-...", "borrower_name": "Asha", "borrower_contact": "98400 00000",
  "listing": { "title": "...", "image_url": "...", "area": "Adyar",
    "owner_name": "Priya", "owner_contact": "..." } } } // contact revealed on
confirmation

409 Conflict
{ "error": "Someone just booked those dates – please pick different ones." }
  
```

The borrower's name comes from their account (`users.name`), never from the request body. Related read endpoints:

- `GET /api/listings/:id/availability` → `{ "booked": [ { "start_date", "end_date" } ] }`: upcoming confirmed bookings (`end_date ≥ today` in IST) with no borrower details. Public; used to grey out the calendar.
- `GET /api/bookings/mine` `auth` → `{ "bookings": [ ... ] }`: the signed-in borrower's bookings ordered by `start_date` (My Rentals).

8.4 Mock payment, and how to make it real

`api/_lib/payments.ts` exports one function. Today it validates the amount and returns a reference like `MOCK-UPI-9F2C41A07B3E`; no money moves, and the UI labels it as a demo payment.

```

export async function chargeAdvance(input: { amount: number; method: 'upi' | 'card' }): Promise<{ ref: string }>
  
```

To go live, replace the body with a real provider, for example a Razorpay order created server-side plus verification of the payment signature with the provider's secret, and return the provider's payment id as `ref`. The rest of the flow does not change. One thing to design deliberately when you do: today the charge happens *before* the insert, so a 409 after a real capture would need a refund or void. The usual fix is to authorise first and capture only after the insert succeeds, or to insert a `pending` row first.

9 Accounts, sessions and Google sign-in

NEW DESIGN

DRAPE no longer uses Supabase Auth for accounts or sessions (it only brokers Google sign-in, Section 9.7). Accounts are rows in `public.users`, passwords are hashed by the API with Node's `scrypt`, and sessions are opaque random tokens stored hashed in `public.sessions` and carried in an `httpOnly` cookie. Listings, bookings and notifications reference `public.users`. The browser never holds a readable token.

9.1 Tables (migrations `...0930000004_own_auth.sql`, `...0930000006_login_attempts.sql`)

```
create table public.users (
  id          uuid primary key default gen_random_uuid(),
  created_at  timestampz not null default now(),
  email       extensions.citext not null unique check (email ~ '...@...'),
  name        text not null check (length(name) between 1 and 80),
  password_hash text,          -- scrypt (or legacy bcrypt); null = Google-only
  google_sub   text unique     -- Google's stable account id, once linked
);
create table public.login_attempts ( -- ...0930000006_login_attempts.sql
  email       extensions.citext primary key, -- any email, registered or not
  failed      int not null default 0,
  locked_until timestampz,
  updated_at  timestampz not null default now()
);
create table public.sessions (
  token_hash text primary key, -- hex SHA-256 of the cookie token
  user_id    uuid not null references public.users(id) on delete cascade,
  created_at timestampz not null default now(),
  expires_at timestampz not null
); -- indexes: sessions_user_idx (user_id), sessions_expiry_idx (expires_at)
```

All three tables follow the house rule: RLS on, no policies, service role only. `login_attempts` has no foreign key on purpose: it counts attempts per *email address*, including addresses with no account. Migration `...0006` also dropped the earlier per-user `users.failed_logins` and `users.locked_until` columns. The `own_auth` migration copies existing Supabase Auth accounts into `public.users` **with the same ids** and their bcrypt hashes (`auth.users.encrypted_password`), re-points the `listings.owner_id`, `bookings.borrower_id` and `notifications.user_id` foreign keys at `public.users`, and drops the Phase 7 `profiles` table and its `on_auth_user_created` trigger. Keeping the ids means every existing listing, booking and notification stays attached to the right person.

9.2 Password hashing (`api/_lib/passwords.ts`)

- `scrypt` (`node:crypto`) with a fresh random **16-byte salt** per password, parameters `N=16384`, `r=8`, `p=1`, 64-byte key. The stored string is self-describing, `scrypt$16384$8$1$<salt b64>$<hash b64>`, so the cost can be raised later: a hash made with a different `N` is flagged `needsRehash` and upgraded at the next login.
- Verification recomputes the hash with the stored parameters and compares with `crypto.timingSafeEqual`, so comparison time does not leak how many bytes matched. Passwords must be 8–200 characters.
- **Migrated accounts.** Two teammates' older accounts came from Supabase Auth with **bcrypt** hashes (`$2a$/$2b$/$2y$`). `verifyPassword()` checks those with `bcryptjs` and returns `needsRehash: true`; the login route then stores a fresh `scrypt` hash. After one login the account is indistinguishable from any other.

9.3 Sessions (`api/_lib/sessions.ts`)

- On sign-up or login, `createSession()` generates a **32-byte random token** (base64url), inserts `token_hash = SHA-256(token)` with `expires_at = now + 30 days`, deletes that user's already-expired sessions, and returns the cookie:

```
Set-Cookie: drape_session=<token>; Path=/; Max-Age=2592000; HttpOnly; SameSite=Lax; Secure
```

`Secure` is omitted only for plain- `http localhost / 127.0.0.1`, so local development works.

- Every request** that needs a user resolves it server-side: `sessionUserId()` reads the cookie, hashes it, looks up the session and rejects it if `expires_at` has passed; `getUser()` in `api/_lib/auth.ts` then loads `id`, `email`, `name` from `users`. `requireUser()` throws `401 "Please sign in first."`. Handlers never accept a user id from the client.
- Storing only the hash means a leaked database backup contains no usable session tokens.
- Logout** (`POST /api/auth/logout`) deletes the session row and returns an expired cookie (`Max-Age=0`), so the token is dead server-side even if a copy exists somewhere.

9.4 Brute-force and enumeration protection (`api/_routes/auth/login.ts`)

- Lockout, per email address.** Each miss upserts `public.login_attempts` for that email and increments `failed`, whether or not an account exists. The 5th (`MAX_FAILED_LOGINS`) sets `locked_until = now + 10 min ( LOCK_MINUTES )` and resets the counter. The lock is checked first; while locked, login returns `429 "Too many attempts for this email – try again in N min."`. A successful login deletes the email's row.
- No email enumeration.** An unknown email, and a Google-only account with no password, both run `burnPasswordCheck()`, a full script verify against a dummy hash, so the response time matches a wrong password, and all three return the same `401 "Wrong email or password."`. Because the lockout is keyed by email rather than by account, a `429` does not reveal whether an account exists either.

9.5 CSRF defence

Because the session is a cookie, the API must make sure other sites cannot trigger state-changing requests with it. Three layers work together:

- SameSite=Lax** cookies are not sent on cross-site POSTs.
- JSON-only POST bodies.** `readJson()` rejects any body whose `Content-Type` is not `application/json` with `415`. An HTML form on another site cannot send that type without a CORS preflight, which the API never grants.
- Origin check** in the router itself: `crossSite()` in `api/router.ts` runs before route matching. A non-GET request whose `Origin` host differs from the request host (`x-forwarded-host / host`) gets `403 {"error":"cross_site_request"}`. Requests with no `Origin` (server-side scripts) and GET/HEAD are unaffected.

9.6 Endpoints

ENDPOINT	BODY / BEHAVIOUR
<code>POST /api/auth/signup</code>	<code>{ name, email, password }</code> (email lower-cased, password ≥ 8) $\rightarrow$ <code>201 { user: { id, email, name } }</code> + session cookie. <code>409</code> if the email already has an account.
<code>POST /api/auth/login</code>	<code>{ email, password }</code> $\rightarrow$ <code>200 { user }</code> + session cookie. <code>401 "Wrong email or password."</code> for every miss (including unknown emails and Google-only accounts), <code>429</code> while that email is locked.
<code>POST /api/auth/logout</code>	Deletes the session row, clears the cookie $\rightarrow$ <code>{ ok: true }</code> .

ENDPOINT	BODY / BEHAVIOUR
GET /api/auth/providers	{ password: true, google: boolean }. google is true only when Supabase's /auth/v1/settings reports the Google provider enabled (cached 5 minutes); the UI hides the button otherwise.
GET /api/auth/google	Creates a PKCE verifier cookie and 302-redirects to Supabase Auth's Google authorize URL (or to /?auth_error=google_unavailable).
GET /api/auth/google/callback	Exchanges ?code= with the verifier; links/creates the DRAPE user, 302 to /?signed_in=google with a session cookie (or /?auth_error=google_failed google_unverified).
GET /api/me auth	{ user: {id, email, name}, stats: { listed, rentals, earned } }. earned sums total of confirmed bookings of the user's listings.

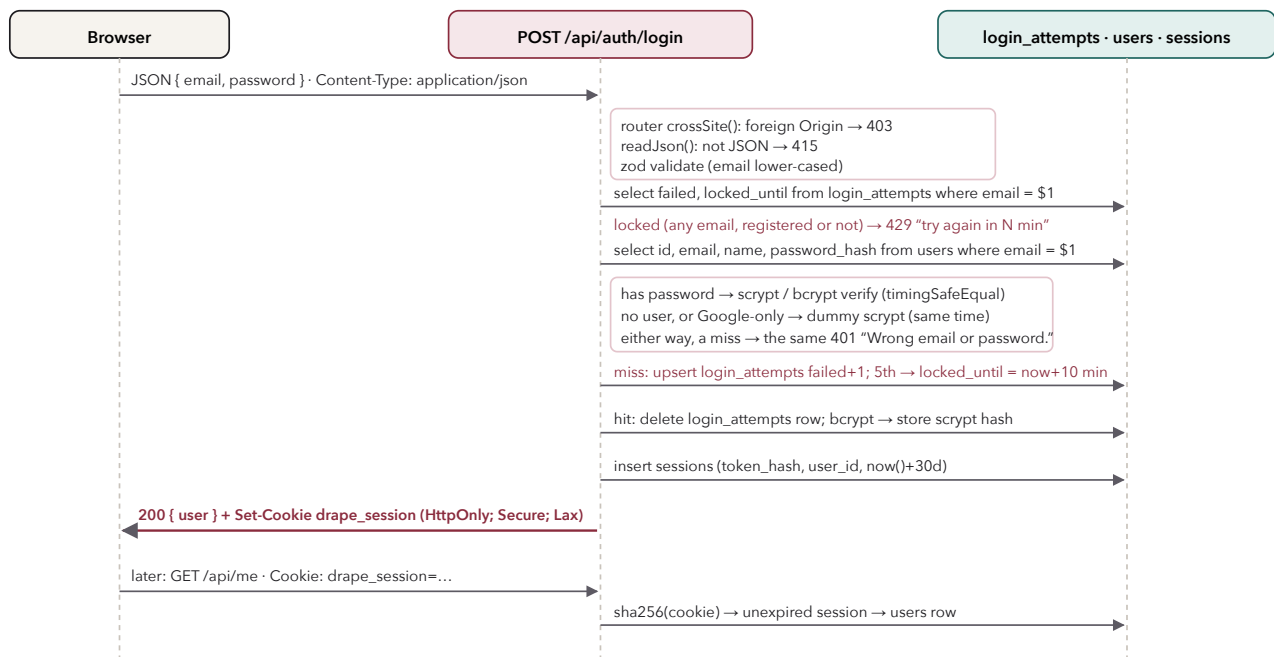


Figure 7. Email + password login. Registered, unregistered and Google-only emails follow the same path and get the same responses; only a correct password differs. Below: how every later request resolves the user.

9.7 Google sign-in (Supabase Auth as a PKCE broker)

Google is a secondary option, and it is the **only** place Supabase Auth still appears. Supabase's Google provider runs the Google consent flow for us; DRAPE then takes the verified identity, links or creates a row in **its own** `public.users` table, and starts **its own** `drape_session` cookie. The Supabase session that comes back is discarded, so `auth.users` acts purely as a Google broker. Google's client ID and secret are configured in the Supabase dashboard, not in DRAPE's environment.

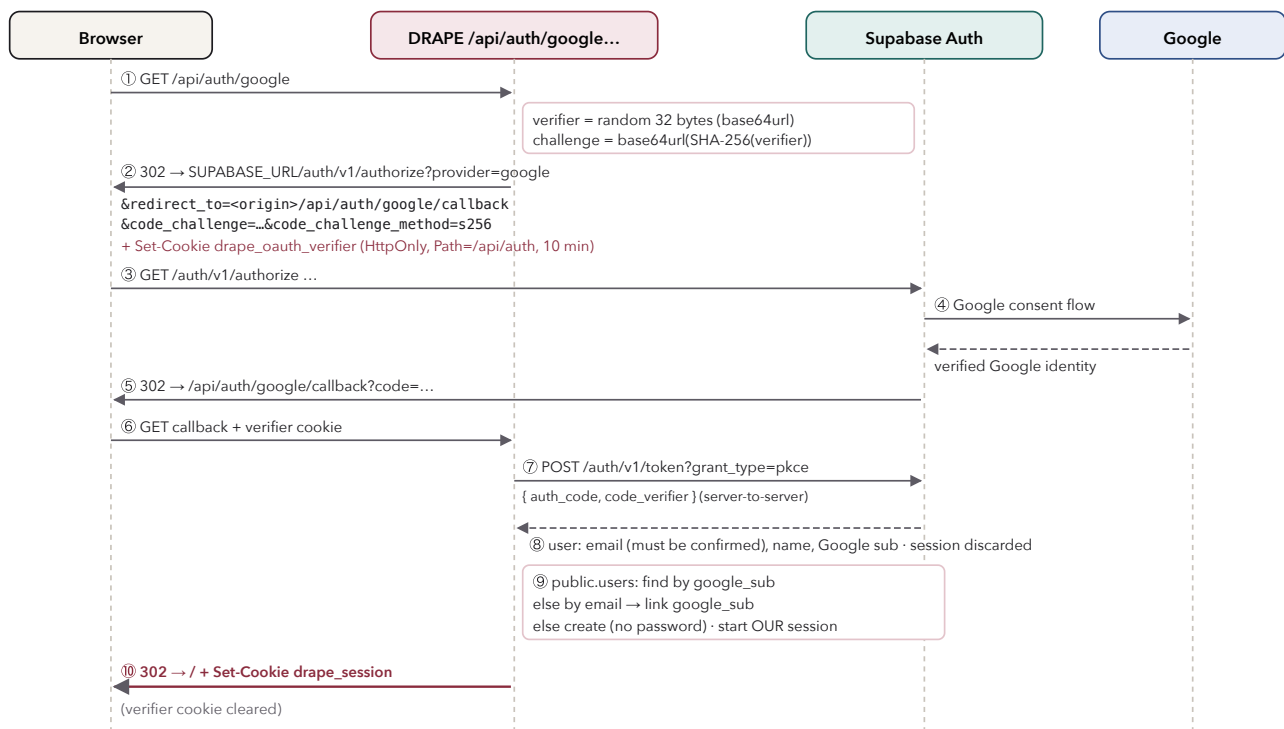


Figure 8. Google sign-in with PKCE. Only the browser that started the flow holds the verifier cookie, and the code is useless without the verifier, so a code intercepted or injected from elsewhere cannot be redeemed. This cookie replaces a separate OAuth `state` cookie.

Why PKCE, and what it protects

- **Binding to the browser.** `drape_oauth_verifier` is `httpOnly`, scoped to `Path=/api/auth`, and lives 10 minutes. The token exchange at step ⑦ only succeeds with the verifier whose SHA-256 matches the challenge sent at step ②, so the callback is tied to the browser that started it (login-CSRF and code-injection protection).
- **No tokens in the browser.** The code exchange is server-to-server; the Supabase access/refresh tokens are read once for the user's identity and then thrown away.
- **Verified identities only.** `exchangeCode()` in `api/_lib/google.ts` posts `{ auth_code, code_verifier }` with the service-role `apikey`, requires a `google` identity and a confirmed email (`email_confirmed_at`), else the callback redirects to `?auth_error=google_unverified`.
- **Linking.** The user is matched by `google_sub` (the Google identity's `sub`) first, then by email, in which case `google_sub` is attached to the existing row; otherwise a new user without a password is created. Such an account can only sign in with Google; a password login for it gets the generic `401`.

Configuration (Supabase dashboard, not env vars)

- **Authentication** → **Providers** → **Google**: client ID and secret from Google Cloud, with the Google-side authorised redirect URI `https://<project>.supabase.co/auth/v1/callback`.
- **Authentication** → **URL Configuration** → **Redirect URLs**: DRAPE's `/api/auth/google/callback` on production and on localhost, so Supabase will redirect back to it.
- `GET /api/auth/providers` returns `google: true` only when Supabase's public auth settings report the Google provider as enabled; otherwise the UI hides the button.

10 Notifications

Owners learn about bookings through the bell in the header. The design goal: a booking can never exist without its notification. That is why the notification is written by a **trigger**, inside the same transaction as the booking insert, rather than by a second API call that could fail halfway.

```
create or replace function public.notify_owner_of_booking()
returns trigger language plpgsql security definer set search_path = '' as $$
declare owner uuid;
begin
  select owner_id into owner from public.listings where id = new.listing_id;
  if owner is not null and new.status = 'confirmed' and owner is distinct from new.borrower_id then
    insert into public.notifications (user_id, type, booking_id) values (owner, 'booking_received',
new.id);
  end if;
  return new;
end $$;
create trigger on_booking_created after insert on public.bookings for each row execute function
public.notify_owner_of_booking();
```

No notification is created for unowned (seed) listings, for non-confirmed rows, or when an owner books their own piece (the API also blocks that with a 400). If the insert fails, for example on the no-overlap constraint, the notification is rolled back with it.

10.1 Reading and clearing

```
GET /api/notifications                                // signed in; newest 30
200 { "notifications": [ { "id", "created_at", "read_at": null, "type": "booking_received",
  "booking": { "id", "start_date", "end_date", "days", "total", "advance", "status",
    "borrower_name", "borrower_contact",
    "listing": { "id", "title", "image_url", "area" } } } ],
  "unread": 1 }

POST /api/notifications/read { "ids": ["..."] } // ids optional (max 100); omit to mark all as read
200 { "ok": true }
```

The query is always scoped by `user_id` = the session user, and the update only touches that user's unread rows (`read_at is null`), so one user cannot mark or read another's notifications.

10.2 Delivery: polling

There is no websocket or push. While signed in, `src/App.tsx` calls `GET /api/notifications` immediately, then every **30 seconds**, and whenever the tab regains focus. Worst-case latency is therefore about 30 s, which is fine for “someone booked your lehenga”. Moving to Supabase Realtime or web push later would not change the trigger or the table.

11 AI providers and rate limits

TASK	PROVIDER · MODEL	TIMEOUT	WHERE / NOTES
Photo → tags	Gemini <code>gemini-3.5-flash-lite</code> (<code>GEMINI_VISION_MODEL</code>)	20 s	<code>vision.ts</code> . Minimal thinking measured about 3 s; the full flash model had multi-second variance (one 82 s outlier).

TASK	PROVIDER · MODEL	TIMEOUT	WHERE / NOTES
Embeddings	Gemini <code>gemini-embedding-001</code> , 768-d	10 s	<code>gemini.ts embed()</code> . Locked: changing it means re-embedding every listing and re-calibrating the floor.
Query parser	OpenAI <code>gpt-5.4-mini</code> (<code>OPENAI_TEXT_MODEL</code>), reasoning off	5 s	<code>text-llm.ts</code> . Measured about 0.9–1.2 s. Falls back to Gemini <code>GEMINI_TEXT_MODEL</code> .
“Why it fits”	same as parser	6 s	One batched call for up to 6 results.

11.1 Why OpenAI handles search text

The Gemini free tier allows **15 requests per minute per model**. Uploads need one vision call and one embedding; each search needs an embedding plus two text calls (parser and reasons). If search text also used Gemini, a handful of searches in one minute could block uploads, and vice versa. So the search-time text calls go to OpenAI, which has separate and much higher limits, and Gemini is reserved for vision and embeddings.

`textJson()` tries OpenAI (Chat Completions, `response_format: json_schema` with `strict: true`) when `OPENAI_API_KEY` is set. If the key is missing, or OpenAI errors or times out, it logs a warning and calls Gemini with the equivalent OpenAPI-subset schema. Each prompt therefore carries two schemas describing the same shape (`PARSER_JSON_SCHEMA` / `PARSER_GEMINI_SCHEMA`, and likewise for reasons).

11.2 Failure behaviour, by call

CALL	IF IT FAILS...
Vision	One retry (not for 429). Then 502 <code>analysis_failed</code> with the stored photo → manual tagging.
Embedding	No fallback: publishing or searching cannot proceed without a vector. A 429 surfaces as <code>rate_limited</code> ; other errors as 502.
Parser	OpenAI → Gemini, with one retry around the whole step. Then degrade to raw-text semantic search.
Reasons	OpenAI → Gemini. Then results without reasons.

11.3 429 handling

When Gemini returns HTTP 429, `call()` in `gemini.ts` reads the “retry in N s” hint from the error message (default 30 s) and throws `RateLimitError(N)`. `withOneRetry()` deliberately does not retry it. `route()` turns it into:

```
HTTP/1.1 429 Too Many Requests
retry-after: 17
{ "error": "rate_limited", "message": "AI is busy – try again in 17s", "retry_after": 17 }
```

The UI shows “try again in N seconds”. The seed and retag scripts read `retry_after`, sleep, and retry (up to 5–6 times), which is how 79 listings were seeded on the free tier.

12 Security summary

CONCERN	DEFENCE (AND WHERE)
Secrets	All keys are read from <code>process.env</code> inside <code>/api</code> only, and never prefixed with <code>VITE_</code> (which would bundle them into the browser). <code>requireEnv()</code> fails closed with a 500 if one is missing. <code>GET /api/health</code> reports only booleans.

CONCERN	DEFENCE (AND WHERE)
Direct database access	RLS on with no policies on every table; RPC execute revoked from <code>public/anon/authenticated</code> . Only the service-role client in <code>api/_lib/supabase.ts</code> can read or write.
Private fields	<code>PUBLIC_LISTING_COLUMNS</code> excludes <code>owner_contact</code> , <code>embedding</code> and <code>location</code> . Contact comes only from <code>/api/listings/:id/contact</code> or a booking confirmation. Availability returns dates only. A test asserts the list endpoint never selects <code>owner_contact</code> .
Identity	The user is always resolved server-side from the <code>drape_session</code> cookie. User ids, borrower names, prices and advances in request bodies are never trusted.
Passwords	scrypt (N=16384, r=8, p=1) with per-password 16-byte salt; minimum 8 characters; per-email lockout in <code>login_attempts</code> (5 misses → 10 min) that treats unknown emails the same; <code>timingSafeEqual</code> ; bcrypt upgraded on login; 5-strike 10-minute lockout; dummy hash for unknown emails.
Session tokens	32 random bytes; only SHA-256 stored; 30-day expiry; cookie <code>HttpOnly</code> (no JavaScript access, so XSS cannot steal it), <code>Secure</code> , <code>SameSite=Lax</code> ; logout deletes the row.
CSRF	<code>SameSite=Lax</code> + JSON-only POST bodies (415 otherwise) + router-level <code>Origin</code> check on every non-GET request (403 <code>cross_site_request</code>). Google sign-in uses PKCE: the verifier sits in a 10-minute <code>httpOnly</code> cookie (<code>drape_oauth_verifier</code> , <code>Path=/api/auth</code>).
Admin operations	<code>x-admin-key</code> header compared to <code>SUPABASE_SERVICE_ROLE_KEY</code> with <code>timingSafeEqual</code> (<code>api/_lib/admin.ts</code>). Requests without the header never touch the key. Used only by scripts: <code>POST /api/admin/retag</code> , <code>POST /api/listings/analyze</code> (seed and readiness scripts) and owner-less seeding via <code>POST /api/listings</code> .
Uploads	Signed-in users (or the admin key) only; 4 MB cap; type from magic bytes, not the client's MIME; random server-chosen object names; bucket allows only image types, 8 MB max.
Image URLs	<code>isOwnImageUrl()</code> : listings may only reference this project's <code>listings</code> bucket, and never a path containing <code>..</code> . Prevents hot-linking arbitrary or hostile URLs.
Input validation	zod on every body and query (<code>shared/contracts.ts</code> , <code>shared/booking-schema.ts</code>); UUIDs validated before any DB call; LLM output validated against the same vocabulary.
Integrity under races	Exclusion constraint for bookings; notification in the same transaction via trigger; <code>security_definer</code> functions pin <code>search_path = ''</code> .
Browser hardening	CSP, <code>X-Frame-Options: DENY</code> , <code>nosniff</code> , a strict Referrer-Policy and a Permissions-Policy on every response (<code>vercel.json</code> , Section 13.4). The CSP's <code>connect-src 'self'</code> means the page can only call its own <code>/api</code> .
Caching	Only public, non-personal reads send <code>public</code> , <code>s-maxage</code> ; anything tied to a user, a session or today's date stays <code>no-store</code> , so the CDN never serves one person's data to another.
Error leakage	<code>route()</code> returns messages for expected errors only; unexpected errors are logged and returned as <code>internal_error</code> .

13 Operations

13.1 Environment variables (names only)

Set them in `.env.local` for `npm run dev` (template: `.env.example`) and in Vercel → Project → Settings → Environment Variables for deployments. All are server-side.

VARIABLE	REQUIRED	PURPOSE
<code>SUPABASE_URL</code>	yes	Project URL; also the prefix for the image URL allow-list.
<code>SUPABASE_SERVICE_ROLE_KEY</code>	yes	Server database/storage access; also the admin key for scripts.

VARIABLE	REQUIRED	PURPOSE
<code>GEMINI_API_KEY</code>	yes	Vision and embeddings (and text fallback).
<code>OPENAI_API_KEY</code>	recommended	Search parser and reasons. Without it, search uses Gemini and shares its quota.
<code>GEMINI_VISION_MODEL</code> , <code>GEMINI_TEXT_MODEL</code> , <code>OPENAI_TEXT_MODEL</code>	optional	Model overrides.
<code>MATCH_FLOOR</code>	optional	Search quality floor (default 0.68).

Google sign-in needs no DRAPE env vars: its client ID and secret live in the Supabase dashboard, and the API reaches Supabase Auth through `SUPABASE_URL`. The Supabase Auth-era browser variables (`VITE_SUPABASE_URL` , `VITE_SUPABASE_PUBLISHABLE_KEY`) are not needed by the new session design.

13.2 Scripts

COMMAND	WHAT IT DOES
<code>node --env-file=.env.local scripts/seed-listings.mjs <manifest.json> [--base URL] [--dry-run]</code>	Seeds through the real pipeline with the admin key: analyze each photo, apply optional tag <code>overrides</code> (the human review step), then publish with the admin key (no owner). Waits out 429s using <code>retry_after</code> .
<code>node --env-file=.env.local scripts/retag-listings.mjs [--base URL] [--only cat1,cat2] [--dry-run]</code>	After a vocabulary change: calls <code>POST /api/admin/retag {id}</code> once per listing (one per request keeps each call inside the function time limit). Re-runs vision on the stored photo and re-embeds; keeps owner, size, price, area and photo. Prints before → after category.
<code>node --env-file=.env.local scripts/verify-prd.mjs [URL] [--area Adyar]</code>	Demo-readiness check (run with <code>--env-file=.env.local</code> so it can send the admin key to analyze): runs the 9 PRD §11 success criteria against a deployment (upload < 10 s, non-clothing rejected, newest listing findable, search p50 < 4 s, filters never violated unless announced, demo query top 3 plausible, fallback notice, ≥ 3 listings per occasion, ≥ 40 listings / 6 occasions / 5 areas). Exits non-zero on failure.
<code>node --env-file=.env.local scripts/create-demo-owner.mjs [email] [password] ["Name"]</code>	Creates or reuses the demo owner in <code>public.users</code> (default <code>owner@drape.demo</code> , generated password printed once, stored as the same <code>scrypt\$...</code> format the API writes) and assigns it every listing with no owner, so bookings on seeded pieces notify someone you can sign in as.

13.3 Tests

`npm test` runs Vitest over `tests/`. External services are mocked (`tests/fake-db.ts` stands in for supabase-js), so the suite runs offline.

FILE	COVERS
<code>api-plugin.test.ts</code>	Router matching (static before <code>:id</code>), rewrite reconstruction keeps query + body, every route file registered, exactly one function, admin-key check.
<code>auth.test.ts</code>	scrypt hashing and random salts, bcrypt verify + upgrade, dummy-hash timing, cookie flags, token hashing, cross-site blocking, signup/login/lockout routes, PKCE challenge and missing-verifier callback, demo-owner hash format.
<code>http.test.ts</code>	<code>route()</code> error mapping.
<code>contracts.test.ts</code>	Vision, listing and parser schemas accept PRD examples and reject off-vocabulary values.
<code>upload-pipeline.test.ts</code>	Magic-byte sniffing, own-bucket URLs, vision normalisation, embedding text excludes size/price, analyze outcomes (200/422/502/415), publish computes location (lng lat) and a 768-d embedding and ignores client-sent ones.
<code>search.test.ts</code>	Chip override, ladder order and skipping, size/gender never relaxed, match floor, relaxed messages, reasons clamping.

FILE	COVERS
<code>bookings.test.ts</code>	Quote maths, IST “today”, date validation, overlap semantics, POST /api/bookings (server recompute, 401, own listing, 409, 404), account-scoped My Rentals and notifications.
<code>listings-api.test.ts</code>	Public column list, occasion filter and limit, 404s, contact only from the contact route.
<code>outfits.test.ts</code> , <code>esm-imports.test.ts</code> , <code>ui-lib.test.ts</code>	Slot rules; <code>.js</code> import suffixes for Vercel ESM; UI helpers.

13.4 Deploy

- `vercel deploy --prod .` `vercel.json` pins the function to `bom1` (Mumbai), next to the database. `npm run build` type-checks (`tsc -b`) and builds the SPA.
- **Hobby plan: max 12 functions per deployment.** DRAPE deploys exactly one (`api/router.ts`); route files live under `api/_routes/` where the underscore keeps Vercel from deploying them. Vercel also rejects bodies over 4.5 MB, hence the 4 MB image cap.
- **Edge headers and caching** (`vercel.json` `headers`). Every response gets a Content-Security-Policy (`default-src 'self'` ; images only from self, `data: / blob: , *.supabase.co` and Unsplash; fonts from Google Fonts; `connect-src 'self'` ; `frame-ancestors 'none'`), `X-Frame-Options: DENY` , `X-Content-Type-Options: nosniff` , `Referrer-Policy: strict-origin-when-cross-origin` and a `Permissions-Policy` that turns off camera, microphone, geolocation and payment. Hashed build output under `/assets/` is cached for a year (`immutable`) and the hero image for a week. Public API reads are cached at the CDN for 30 s to 1 h (Section 3.1).
- New database: apply `supabase/migrations/*.sql` in order (currently `...0929000001` through `...0930000006`).

13.5 Known limitations

- **Negation is not understood.** “No sarees” cannot exclude sarees; semantic search has no “not”. The reasons line does call out the mismatch.
- **Occasion is not a hard filter**, only a ranking signal via the embedding.
- **Free-tier quotas.** Gemini at 15 RPM per model limits upload bursts. Note that `GEMINI_VISION_MODEL` and `GEMINI_TEXT_MODEL` default to the same model, so by default the Gemini text fallback (used when OpenAI is missing or failing) shares the vision quota. This is deliberate (the separate-quota model tried instead timed out); set `GEMINI_TEXT_MODEL` to change it.
- **Orphaned photos.** Analyzed-but-never-published uploads (and `verify-prd` runs) leave files in Storage. README.md has a SQL query that lists them.
- **Payments are mocked**, and the charge precedes the insert (see 8.4).
- **No cancellation endpoint yet**, although the schema supports `status = 'cancelled'` .
- **Notifications poll** every 30 s; no push.
- **Location is area-level:** 15 fixed centroids, no real geolocation; everything in one area is 0 km apart.
- The contact-reveal endpoint is public by design (the free contact reveal), so it is not rate-limited per user.

14 Reference

14.1 Endpoint reference

auth needs a `drape_session` cookie · **admin** needs `x-admin-key` · all others are public. All bodies are JSON unless noted. Every error is `{ "error": ... }`.

ENDPOINT	INPUT	SUCCESS RESPONSE · NOTABLE ERRORS
GET <code>/api/health</code>	–	<code>{ ok, supabase, gemini }</code> (booleans)
GET <code>/api/areas</code>	–	<code>{ areas: [{ name, lat, lng }] }</code> · CDN 1 h
GET <code>/api/listings</code>	<code>query</code> <code>area?</code> , <code>occasion?</code> , <code>limit</code> 1–48 (default 8), <code>offset</code> 0–10000	<code>{ listings[], total }</code> : nearest first with <code>distance_km</code> when <code>area</code> is set, else newest first · 400 bad occasion/area · CDN 30 s
POST <code>/api/listings/analyze</code> auth / admin	multipart field <code>image</code> ≤ 4 MB	<code>{ image_url, draft }</code> · 401, 413, 415, 422 not_clothing, 429, 502 analysis_failed
POST <code>/api/listings</code> auth / admin	<code>draft</code> + <code>image_url</code> , <code>size</code> , <code>price_per_day</code> , <code>area</code> , <code>owner_name</code> , <code>owner_contact</code>	201 <code>{ listing }</code> · 400 foreign image / unknown area / invalid
GET <code>/api/listings/:id</code>	uuid	<code>{ listing }</code> (public columns) · 404 · CDN 60 s
GET <code>/api/listings/:id/contact</code>	uuid	<code>{ owner_name, owner_contact }</code> · 404
GET <code>/api/listings/:id/availability</code>	uuid	<code>{ booked: [{ start_date, end_date }] }</code>
GET <code>/api/listings/:id/complete</code>	uuid	<code>{ slot, pairs_with, results[] }</code> (≤ 8, each with <code>slot</code>) · 404 · CDN 60 s
POST <code>/api/search</code>	<code>{ query ≤ 500, area, size?, max_price?, reasons? }</code>	<code>{ parsed, relaxed, results[] }</code> + <code>Server-Timing</code> · 400 unknown area
POST <code>/api/search/reasons</code>	<code>{ query, ids: uuid[1..6] }</code>	<code>{ reasons: [{ id, reason }] }</code>
POST <code>/api/bookings</code> auth	<code>{ listing_id, start_date, end_date, borrower_contact, payment_method: upi card }</code>	201 <code>{ booking }</code> · 400 dates/own listing, 401, 404, 409 overlap
GET <code>/api/bookings/mine</code> auth	–	<code>{ bookings[] }</code>
GET <code>/api/notifications</code> auth	–	<code>{ notifications[], unread }</code>
POST <code>/api/notifications/read</code> auth	<code>{ ids?: uuid[] ≤ 100 }</code>	<code>{ ok: true }</code>
GET <code>/api/me</code> auth	–	<code>{ user, stats: { listed, rentals, earned } }</code> · 401
POST <code>/api/auth/signup</code>	<code>{ name, email, password ≥ 8 }</code>	201 <code>{ user }</code> + session cookie · 409 email taken
POST <code>/api/auth/login</code>	<code>{ email, password }</code>	<code>{ user }</code> + session cookie · 401 generic for every miss · 429 email locked

ENDPOINT	INPUT	SUCCESS RESPONSE · NOTABLE ERRORS
POST /api/auth/logout	–	{ ok: true }; session row deleted, cookie cleared
GET /api/auth/providers	–	{ password: true, google }
GET /api/auth/google	–	302 to SUPABASE_URL/auth/v1/authorize?provider=google... + verifier cookie
GET /api/auth/google/callback	query code + verifier cookie	302 to / + session cookie
POST /api/admin/retag admin	{ id }	{ id, before, after } or { id, skipped } · 403, 404

Router-level errors for any path: 403 cross_site_request (non-GET from a foreign Origin), 404 not_found, 405 method_not_allowed; bodies: 415 if not application/json; validation: 400 invalid_request with zod issues; AI quota: 429 rate_limited.

14.2 Glossary

TERM	MEANING IN DRAPE
Anchor	The listing a “Complete the look” request starts from.
Cosine similarity	$1 - (a \leq b)$; how close two embeddings point. Higher is more alike.
Draft	The vision model’s proposed tags for a photo, edited by the lister before publishing.
Embedding	A 768-number vector representing meaning, from gemini-embedding-001. Documents use RETRIEVAL_DOCUMENT, queries RETRIEVAL_QUERY.
Exclusion constraint	A Postgres constraint forbidding two rows whose values conflict under given operators (here: same listing and overlapping dates).
Fallback ladder	The sequence strict → wider radius → no budget → anywhere used when a step has no good match.
HNSW	Hierarchical navigable small-world graph: pgvector’s approximate nearest-neighbour index.
Match floor	MATCH_FLOOR (0.68): the minimum top similarity for a ladder step to count as a hit.
PKCE	Proof Key for Code Exchange: the OAuth client keeps a random verifier and sends only its hash; the code can be redeemed only with the verifier. Used for Google sign-in.
PostGIS / geography	Geo extension; geography(point, 4326) stores lat/lng and measures in metres.
Relaxed message	The sentence explaining which filter the ladder loosened.
RLS	Row-level security. On with no policies means “no access except the service role”.
RPC	A Postgres function called via supabase-js .rpc(): match_listings, complete_the_look.
Service role	The Supabase key that bypasses RLS. Only the API (and trusted scripts) hold it.
Session	A public.sessions row keyed by the SHA-256 of the random token in the drape_session cookie.
Slot	Where a category sits in an outfit: top, bottom, one_piece, layer, other.
Style query	The parser’s rewrite of the request as a description of the ideal outfit; what gets embedded.
Vocabulary	The fixed lists in shared/vocab.ts that models, validators and UI all share.